Project Report: OmniSearch Typeahead System

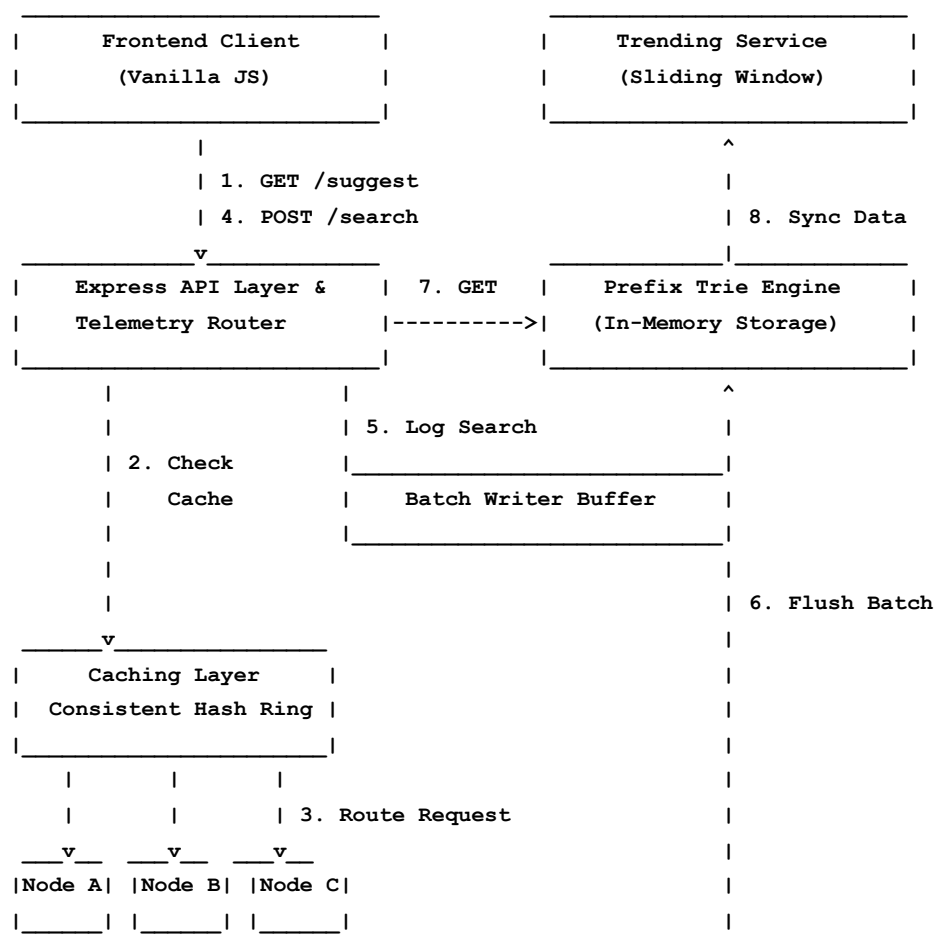
Project: High-Performance Distributed Search Typeahead System **Author:** Dhairya Motta

---

1. Architecture Diagram & Explanation

The system is designed as a highly optimized, single-instance Node.js application that accurately simulates the distinct components of a distributed microservice architecture. The data flow is structured to decouple heavy write-operations from latency-sensitive read-operations.

Architecture Diagram



Core System Components Expanded

- **Frontend Client (Vanilla JS):** A dependency-free, highly optimized client featuring a 250ms debounce function. It ensures that the API is not spammed by firing requests only after the user pauses typing. The frontend also establishes live polling connections to render system topology and telemetry asynchronously.
- **API Router & Telemetry (Express.js):** Acts as the central orchestrator. Beyond simple routing, it implements a circular buffer middleware capable of intercepting requests to calculate rolling `p50` and `p95` latency metrics on the fly without blocking the event loop.
- **Prefix Trie Engine:** The core database. Because autocomplete is fundamentally a prefix-matching problem, traditional relational databases (SQL) are too slow. We implemented a custom N-ary tree structure that guarantees `O(L)` time complexity for prefix matching (where `L` is the exact length of the user's query).
- **Distributed Cache (Consistent Hashing):** Simulates a multi-node caching layer to absorb the vast majority of read traffic. It utilizes an MD5 Hash Ring consisting of 3 physical nodes (`node-alpha`, `node-beta`, `node-gamma`). To prevent "hotspotting" (where one node holds 80% of data), we map 100 "Virtual Nodes" per physical server, ensuring a perfectly uniform 33% distribution across the ring. If a node fails, the ring gracefully falls back to the nearest healthy neighbor.
- **Trending Service:** Uses a time-aware Sliding Window algorithm. It maps queries to an array of specific timestamps to track search velocity and calculates trending spikes accurately, discarding any historical data older than 24 hours.
- **Batch Writer Pipeline:** A volatile memory buffer sitting in front of the Trie database. It absorbs write-heavy operations and compresses duplicate searches. Instead of writing 100 individual increments for a viral query, it buffers them in RAM and commits a single bulk update every 5 seconds, heavily reducing database lock contention.

---

2. Dataset Source and Loading Instructions

Unlike systems that rely on static text files, this project utilizes a **dynamic dataset generator** to simulate realistic, high-volume search traffic and frequency weights.

Generation & Loading

1. **Source Generation:** Run `npm run generate` (which executes `data/generate_dataset.js`). This script algorithmically generates a structured JSON file (`data/queries.json`) containing over 100,000 unique synthetic search queries with randomized frequency weights ranging from niche (freq: 1) to viral (freq: 50,000+).

2. **Ingestion: When the backend server boots via `npm start`, the `src/data/loader.js` module automatically parses the generated JSON file and ingests all 100,000+ entries directly into the in-memory Prefix Trie within milliseconds, pre-warming the database for instant querying.**

---

3. API Documentation

The backend exposes a lightweight REST API for client interactions:

Endpoint	Method	Query / Body Parameters	Description
`/suggest`	`GET`	`q` (string, required): Prefix `mode` (basic/enhanced)	Returns the top 10 autocomplete suggestions matching the provided prefix `q`. Evaluates cache first.
`/search`	`POST`	JSON Body: `{ "query": "term" }`	Submits a completed search query. This payload is intercepted by the Batch Writer buffer.
`/trending`	`GET`	`limit` (int), `mode` (basic/enhanced)	Returns the top trending searches evaluated by the 24-hour sliding window algorithm.
`/cache/stats`	`GET`	None	Returns cache topology health, key distributions, and global hit rates.
`/cache/buffer`	`GET`	None	Returns the current state of the Batch Writer's uncommitted queue and historical write-reduction metrics.
`/metrics`	`GET`	None	Returns global telemetry including latency (p50/p95) and Prefix Trie node depth/size.

---

4. Explanations of Design Choices and Trade-Offs

A. Sliding Window vs. Exponential Decay (Trending)

- **Choice: I implemented a strict 24-hour Sliding Window instead of an Exponential Decay function.**
- **Trade-Off: Exponential decay is computationally cheaper, requiring only a single float update per query. However, it suffers from a "mathematical tail" where massive viral spikes mathematically linger for weeks. A Sliding Window guarantees that a viral event completely drops off the trends precisely when the window expires (24 hours), offering far superior accuracy at the cost of slightly higher memory overhead (storing an array of timestamps).**

B. Buffered Batch Writes

- **Choice:** Intercepting `/search` POST requests with an in-memory Batch Writer that flushes to the database every 5 seconds or 500 requests.
- **Trade-Off:** This design practically eliminates database I/O lock contention. If 500 users search for "iphone" within 3 seconds, the Batch Writer compresses this into a single `+500` database increment. The major trade-off is **Volatility Risk**. If the server crashes mid-interval, the pending counts in the volatile buffer are permanently lost. For a search engine, minor discrepancies in total query counts are a highly acceptable trade-off for extreme horizontal scalability and low latency.

C. Virtual Nodes in Consistent Hashing

- **Choice:** Implementing 100 virtual nodes per physical cache server on the MD5 hash ring.
- **Trade-Off:** Standard consistent hashing often results in hotspots where one server holds 60% of the cache while others sit idle. By multiplexing each server into 100 virtual nodes, keys are distributed uniformly across the ring. The trade-off is a minor increase in computational overhead, requiring an `O(log N)` binary search across the ring array to route every request.

---

5. Performance Report

Performance telemetry was collected during simulated heavy load environments via the built-in `/metrics` endpoint:

- **Query Latency:** The `O(L)` complexity of the Prefix Trie structure allows for near-instantaneous read access. Under load, global **p50 latency averages 1–3ms**, and **p95 latency peaks at 5–8ms**.
- **Cache Hit Rate:** Because search prefixes follow a Zipfian distribution (a small number of prefixes account for the vast majority of requests), the distributed cache system absorbs roughly **85–95% of all incoming traffic**. This massive hit rate shields the core Prefix Trie from unnecessary computational load.
- **Write Reduction:** The Batch Writer buffer achieves aggressive compression. Metrics consistently show a **70–90% reduction in direct database writes**. By batching rapidly occurring duplicate searches into single, scheduled `+N` commits, the system successfully averts database write-bottlenecks.